Ders 14 Genişletilmiş Notu: Proje Test Sunumları II: Birim, Sistem, Kullanıcı ve Performans Testi Örnekleri

Ders 14 Genişletilmiş Notu: Proje Test Sunumları II: Birim, Sistem, Kullanıcı ve Performans Testi Örnekleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Düzeni ve Sunumdan Beklenenler
- 2. Birinci Grup: Dil Kursu Otomasyon Sistemi
- 3. İkinci Grup: İnsan Kaynakları Sistemi
- 4. Üçüncü Grup: Bitirme ve Ara Proje Yönetim Sistemi
- 5. Dördüncü Grup: Seyahat Acentesi Uygulaması
- 6. Beşinci Grup: Otopark Otomasyon Sistemi
- 7. Altıncı Grup: Sherlock Komut Satırı Aracı
- 8. Yedinci Grup: Flask ile Yazılmış Alışveriş Uygulaması
- 9. Sekizinci Grup: Instagram Otomatik Hesap Oluşturma Botu
- 10. Dokuzuncu Grup: Android TV için Leanback Launcher
- 11. Tek Kişilik Proje: Depo Stok Yönetim Sistemi
- 12. Sunumlardan Çıkan Ortak Noktalar
- Örnek Sorular
- Dersin Toparlaması

1. Dersin Düzeni ve Sunumdan Beklenenler

Bu dönemin son dersidir ve yeni bir konu anlatılmadı. Bir önceki derste üç grup dönem projesinin test sunumunu yapmıştı. Bu derste kalan bütün gruplar sunum yaptı: dokuz grup ve tek kişilik bir proje. Her proje, dönem içinde teorik olarak görülen test planı (test plan) ve test sonuç raporu (test report) yapısının gerçek bir yazılım üzerinde uygulanmasıydı.

Hoca derse başlarken süreyi belirledi. Bir önceki haftadan farklı olarak her grubun süresi yaklaşık 15 dakikaydı; bir sonraki gruba en fazla 20 dakika dedi. Sunumun, gruptaki herkes bir bölüm anlatacak şekilde paylaştırılmasını istedi.

Sunumların hepsi benzer bir iskeleti izledi:

- Test edilen yazılımın kısa tanıtımı.
- Gereksinim analizi: fonksiyonel (functional) ve fonksiyonel olmayan (non-functional) gereksinimler.
- Test planı: amaç, test edilecek ve edilmeyecek öğeler, giriş ve çıkış kriterleri, hata derecelendirmesi, roller, takvim, riskler, ortam ve araçlar.
- Test seviyelerine göre örnekler: birim testi (unit test), sistem testi (system test) ve kullanıcı testi (user test). Sistem testi çoğu grupta performans, yük ve güvenlik testlerini kapsadı.

Hoca ilk sunumun ortasında, gereksinim analizinin nasıl anlatılması gerektiğini de söyledi.

> **Hocanın vurgusu:** Proje tanımı hızlı geçilmelidir. Gereksinim analizinde yalnız fonksiyonel ve fonksiyonel olmayan özellikler kısaca sıralanmalıdır. Use case

diyagramları ayrıntılı gösterilmemeli, senaryolardaki maddelerden birkaç örnek verilmelidir.

Sunumlar boyunca hocanın en sık sorduğu şey, birim testi, sistem testi ve performans testi için **hangi aracın (tool)** kullanıldığıydı. Bir grup bu soruya ürünün adıyla cevap verince hoca, sorunun ürünü değil test aracını kastettiğini açıkça belirtti. Bu nedenle bu notta her grup için kullanılan araçlar ayrıca belirtilmiştir.

2. Birinci Grup: Dil Kursu Otomasyon Sistemi

2.1 Yazılımın tanıtımı

Test edilen yazılım C# ile yazılmış bir masaüstü uygulamasıdır. Bir dil kursunun bütün şube bilgilerini ve şubelerdeki öğrencilerin, öğretmenlerin, derslerin ve sınıfların kayıtlarını tek bir programda, anlaşılır bir arayüzle tutmayı amaçlar. Sisteme şube adı ve şifreyle giriş yapılır; ayrıca bir yönetici girişi vardır. Girişten sonra şube, ders, öğrenci, öğretmen ve sınıf işlemleri için ayrı ekranlar açılır. Grup, sistemin öğrencilerin kayıt oldukları dersleri ve borç durumlarını doğru saklama ihtiyacını da karşıladığını söyledi.

2.2 Gereksinim analizi

Grup fonksiyonel gereksinimleri use case senaryolarıyla anlattı. Örnek senaryo öğrenci silmeydi:

- **Aktör:** yönetici, şube görevlisi ya da sistemi yapan kişinin müdahalesi.
- **Ön koşul:** yönetici veya şube görevlisi sisteme giriş yapmış olmalıdır.
- **Son koşul:** öğrenci sistemden silinmiş olur.
- **Ana başarı senaryosu:** kullanıcı öğrenci işlemleri menüsüne tıklar ve silme işlemine kadar belirli adımları izler.
- **Alternatif senaryo:** silinmek istenen öğrenci sistemde yoksa “böyle bir öğrenci yok” uyarısı verilir.

İkinci örnek öğrenci arama senaryosuydu. Hoca bu noktada açıklama yapmadan fonksiyonel gereksinimlere, ardından fonksiyonel olmayan gereksinimlere geçilmesini istedi. Fonksiyonel olmayan gereksinimler şöyle sıralandı:

- **Kullanıcı arabirimi gereksinimi:** arayüz kullanıcı dostu, sezgisel ve kolay kullanılır olmalıdır.
- **Verimlilik gereksinimi:** sistem gerçek zamanlı çalışmalıdır. Kullanıcının tetiklediği, veritabanındaki bilgiler üzerindeki herhangi bir işlem 10 saniyeden kısa sürede tamamlanmalıdır.
- **Kullanılabilirlik ve sürdürülebilirlik gereksinimi:** sistem otomatik yedek alabilmeli; yetkili kullanıcılar verileri mevcut bir yedekten geri yükleyebilmelidir.
- **“Güvenilirlik” gereksinimi:** sistemin belirli özelliklerini kullanmak için kullanıcının ad ve şifreyle kimliği doğrulanmalıdır. Doğru oturum bilgisi verilmezse erişime izin verilmez.

Son madde üzerine hoca sunumu durdurdu. Anlatılan şeyin güvenilirlik değil güvenlik olduğunu söyledi.

> **Hocanın vurgusu:** Güvenilirlik (reliability) ile güvenlik (security) ayrı kavramlardır. Kimlik doğrulama bir güvenlik gereksinimidir. Burada bir kavram hatası yapılmıştır.
>
> **Ek bilgi (derste anlatılmadı):** Kalite modellerinde (örneğin ISO/IEC 25010) güvenilirlik büyük olasılıkla yazılımın belirli koşullarda, belirli bir süre boyunca hatasız çalışabilme yeteneği olarak tanımlanır; olgunluk, erişilebilirlik, hata toleransı ve kurtarılabilirlik gibi alt özellikleri vardır. Güvenlik ise bilgiyi yetkisiz erişime karşı koruma yeteneğidir; gizlilik, bütünlük, kimlik doğrulama ve hesap verebilirlik gibi alt özellikleri vardır. Bu yüzden kimlik doğrulama güvenliğe, otomatik yedek alma ve yedekten geri yükleme ise daha çok güvenilirliğin kurtarılabilirlik boyutuna yakındır.

Grup aynı başlık altında, veritabanının bulunduğu bilgisayara fiziksel erişimin personelle sınırlı olmasını ve belli bir süre işlem yapılmazsa oturumun otomatik kapanmasını da saydı. Hoca test planına geçilmesini istedi.

2.3 Test planı

Testlerin amacı, gereksinim belgesindeki gereksinimlerin yerine getirilip getirilmediğini görmekti. Yazılım kullanıcı girdisine çok bağlı olduğu için grup fonksiyonel gereksinimleri farklı **girdi kombinasyonlarıyla** denedi. Örneğin bir şubeye öğretmen eklenmeden önce öğretmenin hangi dersi hangi saatte vereceği seçilmiş olmalıdır. Saat seçilmiş ama şube seçilmemiş, ya da şube seçilmiş ama verilecek dil seçilmemiş durumları ayrı ayrı denendi ve sistemin hata verip vermediğine bakıldı.

Üç test türü yapıldı:

- **Birim testi:** bir butona basılmasının fonksiyonel karşılığı test edildi. Butonun çağırdığı fonksiyon için test yazıldı.
- **Sistem testi:** CPU, bellek ve veritabanına erişim sıklığı ölçüldü.
- **Kullanıcı testi:** butonlara elle basılarak denendi.

Giriş ve çıkış kriterleri (entry and exit criteria) testlerin başlangıç ve bitiş koşullarıdır. Bu projede giriş kriteri, test edilen tıklama fonksiyonu çalışmadan önce yapılması gerekenlerin yapılmış olmasıydı: gerekli kutucuk işaretlenmiş mi, doldurulması gereken alanlar doldurulmuş mu? Fonksiyon sonlandıktan sonra çıktı kontrol edilir ve test sonlanır. Performans ve kullanıcı testlerinde giriş kriteri programın açılması, çıkış kriteri programın kapanmasıydı.

Grup test sonucunu yalnız gereksinimlere değil, kaynak koda da bakarak değerlendirdi. Gereksinimlerde belirtilmeyen bir durumda programın **beklenen hata mesajını** vermesi başarılı sayıldı. Örneğin yanlış bir giriş yapıldığında belirli bir hata verilmesi gerekir; bu hata veriliyorsa test başarılıdır. Beklenmeyen başka bir hata çıkarsa test başarısızdır.

Test ortamı, yazılımın geliştirildiği ortamla aynıydı: Visual Studio. Birim test aracı olarak NUnit kullanıldı. Bu araç testleri sırayla ya da tek tek çalıştırıp sonuçları ayrıntılı gösterir. Kaynak olarak Microsoft'un resmî belgeleri ve blog yazıları kullanıldı.

> **Ders sonrası not:** Grup NUnit'i "Visual Studio'nun bir aracı" olarak tanıttı. Doğrusu büyük olasılıkla şudur: NUnit, Visual Studio'ya bir test bağdaştırıcısıyla (test adapter) eklenen, açık kaynaklı ayrı bir .NET test çatısıdır. Microsoft'un kendi test çatısı MSTest'tir.
>
> **Ders sonrası not:** Grup, Visual Studio çok kapsamlı bir IDE olduğu için test ve geliştirme ortamının aynı olmasını sorun görmedi. Ortam ayrımı IDE ile ilgili değildir. Ayrılması gereken şey veri, yapılandırma ve kurulumdur; test, geliştiricinin makinesinden bağımsız ve tekrarlanabilir bir ortamda çalışmalıdır.

2.4 Birim testi

Birim testleri gereksinimlere göre senaryolar halinde kuruldu:

- **Öğrenci silme:** silinecek öğrencinin ID'si bir metin kutusuna girilir ve "öğrenci sil" butonuna basılır.
  - Kutu boş bırakılırsa "hata var" değişkeninin `true` olması beklenir.
  - Geçerli bir ID girilirse veritabanına gönderilecek komutun doğru olup olmadığı kontrol edilir.
  - Sayısal olmayan bir değer girilirse yine "hata var" değişkeninin `true` olması ve kullanıcıya hata gösterilmesi beklenir.
- **Öğrenci arama:** geçersiz bir ID girildiğinde veritabanına gönderilecek komutun boş olması beklenir. Ancak komut yine de oluşturulur; bu test başarısız olur. Geçerli ID için komutun doğruluğu kontrol edilir.
- **Öğretmen ekleme:** öğretmenin boş saatleri girildiğinde veritabanına gönderilecek dizginin doğruluğu, öğretmen adı verilmediğinde "hata var" değişkeninin `true` olması kontrol edilir.

Sunumdan önce çalıştırılan 14 testin 12'si başarılı, 2'si başarısız oldu. Başarısız testlerden biri yukarıdaki öğrenci arama testidir. Diğeri ev ve cep telefonu alanlarıdır: bu alanlara sayısal olmayan bir değer girildiğinde hata verilmesi beklenir, ama yazılım bu kontrolü yapmaz.

2.5 Sistem testi ve kullanıcı testi

Uygulama bir masaüstü uygulaması olduğu için grup, seçeneklerinin az olduğunu söyledi ve uygulamayı bütün olarak test etmek için Windows Performance Monitor aracını seçti. Güvenlik ve performans testleri yapıldı; yük testi performans testinin altında ele alındı. Araç bütün thread ve log durumlarını ayrıntılı rapor ediyordu. Sonuçta toplam CPU yükü ve disk erişimi düşük çıktı; diskte saniyede yüzden az okuma-yazma gerçekleşti. Hoca performans testinde hangi aracın kullanıldığını sordu; cevap Performance Monitor oldu.

Kullanıcı testinde önceden belirlenmiş kriterler ve gözlemlerle bir **kullanıcı anketi** hazırlandı. Kullanıcı her özelliği olumlu, olumsuz ya da nötr olarak değerlendirdi. Örneğin "yavaş oturum açma sürecinden rahatsızım" ifadesine kullanıcı olumsuz cevap verdi; buradan uygulamanın yavaş çalışmadığı sonucu çıkarıldı. Kullanıcı testi ayrıca gereksinim bazında yapıldı: her gereksinim için eylemler ve işlem önceliği yazıldı, kullanıcı değerlendirmesini ve yorumunu ekledi. Örneğin silme gereksiniminin önceliği yüksekti; kullanıcı olumlu cevap verdi ve menünün başarıyla açıldığını yazdı. Öğrenci arama ve öğretmen ekleme gereksinimleri de aynı biçimde test edildi.

Güvenlik açısından uygulamada iki kullanıcı türü vardır: şube görevlisi ve yönetici. Şube görevlisi ders ve sınıf ekleyemez; bu işlemler yalnız yöneticiye açıktır.

3. İkinci Grup: İnsan Kaynakları Sistemi

3.1 Yazılımın tanıtımı ve altyapısı

Test edilen sistem, çalışanlarla ilgili bilgileri tutar, çalışanları uygun projelere yerleştirir ve maaş, tazminat, sigorta gibi hesapları yapar. Çalışanların, projelerin ve ekiplerin takibini kolaylaştırarak kaynakların daha verimli kullanılmasını amaçlar.

Grup, testlerin nasıl yapıldığını anlamak için önce altyapıyı tanıttı:

- Programlama dili Java, geliştirme ortamı IntelliJ IDEA. Birim testleri de bu ortamda yapıldı.
- Veritabanı MySQL; sistem localhost üzerinde çalışan bir MySQL sunucusuna bağlanır.
- Arayüz JavaFX ile, Scene Builder ortamında yazıldı; CSS ve FXML kullanıldı.
- Testler JUnit ile yazıldı. Proje Maven ile derlenir.

Proje kapsamında test edilmesi gereken 40 fonksiyonel ve 32 fonksiyonel olmayan gereksinim belirlendi. Fonksiyonel olmayanların 13'ü performans, 13'ü kullanılabilirlik, 6'sı güvenlikle ilgilidir ($13 + 13 + 6 = 32$).

3.2 Test planı

Test planının amacı, kullanıcı rollerinin kendilerinden beklenen davranışları yerine getirdiğini göstermekti. Test edilen modüller çalışan, proje ve takım modülleriydi. Test faaliyetlerinin tamamlanmış sayılması için şu koşullar belirlendi:

- Planlanan yazılım, sistem ve kabul testi faaliyetlerinin hepsi gerçekleştirilmiş olmalıdır.
- Bütün test seviyelerindeki test durumları ve senaryolar başarıyla sonuçlanmış olmalıdır.
- Testlerde ortaya çıkan 1., 2. ve 3. derece hataların tamamı çözülmüş olmalıdır.
- Bütün seviyelerdeki test faaliyetleri raporlanmış olmalıdır.

Hatalar üç dereceye ayrıldı:

- **1. derece:** sistemin genel çalışmasını doğrudan etkileyen hatalar.
- **2. derece:** genel çalışmayı doğrudan etkilemeyen, ama bazı işlevlerin çalışmasını engelleyen hatalar.
- **3. derece:** sistemin çalışmasını etkilemeyen, görsel yönden ortaya çıkan hatalar.

Plan bir Gantt diyagramıyla tamamlandı.

3.3 Birim testi

Her modül kendi içinde test edildi. Toplam 56 JUnit testi yazıldı; testler beklenen değerle dönen değeri karşılaştırır. Örnek testler:

- **Veritabanı bağlantısı:** `DBUtils` adlı test sınıfı veritabanına bağlanılıp bağlanılamadığını kontrol eder. Sunucu açıkken testin geçmesi beklenir.
- **Kullanıcı kaydı:** belirli bir kullanıcı oluşturulup kaydedilir.
- **Giriş:** yetkili ve yetkisiz giriş ayrı ayrı test edilir.
- **Yönetim modülü:** takımları çekme, giriş sayfası ve uyarı (alert) testleri.
- **Proje kontrolü:** veritabanından beklenen değerlerin döndüğü kontrol edilir.

Sunum sırasında testler tek tek gösterildiğinde hepsi geçti. Bütün testler birlikte çalıştırıldığında ise 3 testin başarısız olduğu görüldü.

> **Ders sonrası not:** Canlı gösterimde test sayısı altyazıda karışık geçer: önce "3 tane testin başarısız olduğunu", hemen ardından "43 tanesinin başarısız olduğunu" söyleniyor. Bağlamdan büyük olasılıkla 56 testten 3'ü başarısız, 53'ü başarılıdır.

3.4 Sistem testi ve kullanıcı testi

Sistem testi kapsamında performans, yük ve güvenlik testleri yapıldı. Performans testinde, istenen değerler içinde sistemi kullanabilecek en fazla kullanıcı sayısı belirlendi. Yük testinde belirli bir yük değerinde sistemin tepkisi ölçüldü. Güvenlik testinde güvenlik açığı olup olmadığına bakıldı. Bu testlerin aracı Apache JMeter'dı. Sonuçlar zaman kısıtı nedeniyle sunuma konmadı, rapora konuldu.

Kullanıcı testi için farklı kullanıcı senaryoları oluşturuldu; sunumda dördü gösterildi. İlk senaryo, veritabanı bağlantısı yokken sisteme girmekti: sistem bir uyarı mesajı vererek veritabanına bağlanılmasını istemelidir. İkinci senaryo, yeni bir çalışan ekleyip sistemin ona otomatik olarak uygun bir proje bulmasıydı. Bir takımda en fazla 5 çalışan olabilir. Gösterimde bir takımdan yazılımcı çıkarıldı, ardından yeni bir yazılımcı eklenmek istendi ve sistem bu çalışan için web sitesi geliştirme projesini önerdi. Çalışan eklenirken bilgilerin veritabanı, sunucu ve istemci tarafında tutarlı aktarılması gerekir.

Hoca bu noktada kullanıcı testi yerine birim testinin çalıştırılmasını görmek istedi ve birim testinin yapıldığı ürünün sorulduğunu söyledi. Grup ürünü yani insan kaynakları sistemini anlatmaya başlayınca hoca düzeltti: kastedilen ürün değil, birim testini yaptıkları araçtı. Cevap JUnit oldu. Hoca ardından sistem testi aracı hakkında da bir iki cümle istedi.

Grup JMeter ile örneğin bir kullanıcı için uygulamanın ne kadar sürede açıldığını, açılışta ne kadar bellek ve CPU harcadığını görebildiklerini söyledi. Aracın yüklü olduğu işletim sisteminde olmadığı için canlı gösteremedi. Hoca sonuçlara rapordan bakacağını söyledi.

> **Ders sonrası not:** JMeter büyük olasılıkla esas olarak HTTP, JDBC gibi protokoller üzerinden sunucu tarafına yük bindirmek için kullanılır. Bir masaüstü uygulamasının açılış süresi, CPU ve bellek kullanımı VisualVM gibi bir profil aracıyla daha doğru ölçülür. Grubun anlattığı ölçümler bu yüzden JMeter'ın asıl kullanım alanıyla tam örtüşmüyor.

4. Üçüncü Grup: Bitirme ve Ara Proje Yönetim Sistemi

4.1 Yazılımın tanıtımı

Grup bilgisayar mühendisliği bölümleri için bir proje yönetim sistemi geliştirdi. Çıkış noktası şuydu: Bitirme ve ara projelerinde öğrenciler arasındaki ve öğrenciyle akademisyen arasındaki iletişim e-posta ya da her hocanın kendi tercih ettiği platform üzerinden yürüyor; bu hem yorucu hem dağınık. Sistemde öğrenci proje önerisi (proposal) formunu doğrudan yükler: projenin ara proje mi bitirme projesi mi olduğu, başlığı, özellikleri, kaç kişinin ve kimlerin çalışacağı ve ek dosyalar girilir. Akademisyenler ve asistanlar önerileri görüp onaylar. Ara rapor, bitirme raporu, makale ve sunum gibi sonraki aşamalar da buradan yürütülür. Sistem Django ile yazılmıştır.

4.2 Test edilen gereksinimler

Fonksiyonel gereksinimler arasından test için şunlar seçildi:

- 1 numaralı gereksinim: kullanıcı giriş yapmak için giriş sayfasına girer.

- 4 numaralı gereksinim: akademisyen olarak giriş yapılmışsa kullanıcı proje önerisi yayınlayabilir.
- 14 numaralı gereksinim: ana sayfaya girilince duyurular listelenir.

Bu gereksinimler Django'nun yerleşik `TestCase` sınıfıyla birim testi olarak yazıldı.

4.3 Performans testi

SRS hazırlanırken belirlenen 36 numaralı gereksinim, yazılım eş zamanlı olarak birden fazla kullanıcı tarafından kullanılacağı için potansiyel yükü kaldırabilmesini ister. Bunu sınamak için açık kaynaklı Locust aracı kullanıldı. Locust Python ile kurulur; hangi işlemin yapılacağı bir Python dosyasında tanımlanır. Araç çalıştırıldıktan sonra uygulamanın çalıştığı adres ve sanal kullanıcı sayısı girilir.

Grup, projeye aynı anda en fazla 50 kişinin gireceğini tahmin ettiği için 50 sanal kullanıcıyla ana sayfaya eş zamanlı istek gönderdi. Bu yaklaşık saniyede 8-9 isteğe karşılık geldi. Locust'un kontrol panelinde uygulamanın bu yükü kaldırdığı görüldü; test başarılı oldu.

4.4 Birim testleri

Testler canlı çalıştırıldı: 9 testin hepsi geçti. Testlerin içeriği şöyleydi:

- **Giriş testleri:**
  - Bir kullanıcı oluşturulur. Giriş adresine bu kullanıcının e-posta ve şifresiyle POST isteği atılır. Durum kodunun (status code) başarılı olduğu ve kullanıcının ana sayfaya yönlendirildiği kontrol edilir.
  - Giriş yapıldıktan sonra oturum (session) boyunca aktif kullanıcı bilgisinin bağlamda (context) bulunduğu kontrol edilir.
  - Başarısız durum: alan boş bırakıldığında form hatası (form error) verildiği kontrol edilir.
- **Duyuru testleri:**
  - Ön hazırlık verisi olarak bir öğrenci ve bir koordinatör kullanıcısı ile iki örnek duyuru oluşturulur. Ana sayfaya istek atılır ve sayfanın bu duyuru başlıklarını içerdiği `contains` kontrolüyle doğrulanır.
  - Öğrenci olarak giriş yapılıp duyuru oluşturma sayfasına gidilince, öğrenci duyuru oluşturamayacağı için giriş sayfasına yönlendirilir. Koordinatör ise sayfaya doğrudan erişebilir.
- **Proje önerisi testleri:**
  - Ön hazırlık verisi olarak bir akademisyen ve bir öneri oluşturulur. Önerilerin listelendiği sayfanın mevcut önerileri içerdiği kontrol edilir.
  - Akademisyen olarak giriş yapılıp öneri oluşturma sayfasına POST isteği atılır. İsteğin başarılı olduğu ve öneriler sayfasına yönlendirme yapıldığı kontrol edilir.
  - Yeni öneri koordinatörler onaylamadan ana sayfada listelenmemelidir. Bu `assertNotContains` ile kontrol edilir.
  - Önerinin durumu "accepted" yapılınca öneri artık listede görünmelidir. Bu kez `assertContains` kullanılır.

Bu testler, aynı işlevin hem olumlu hem olumsuz yönden sınanmasına iyi bir örnektir: öğrenci erişememeli, koordinatör erişebilmeli; onaysız öneri görünmemeli, onaylı öneri görünmeli.

5. Dördüncü Grup: Seyahat Acentesi Uygulaması

5.1 Yazılım, ortam ve takvim

Grup, daha önce başka bir derste geliştirdiği bir seyahat acentesi uygulamasını test etti. Uygulama tatil ve ulaşım hizmetlerinin kolayca yönetilmesini, bilet sorgulama, görüntüleme ve satın alma işlemlerinin basit bir arayüzle yapılmasını sağlayan yerel (local) bir sistemdir. C# Windows Forms ile yazılmıştır ve MSSQL kullanır. İki rol vardır:

- **Sistem yöneticisi:** kullanıcı ve firma ekler ve düzenler; tatil, ödeme tipi, yolculuk tipi ve sefer ekler.

- **Müşteri temsilcisi:** bilet satar, tatil rezervasyonu yapar, müşteri bilgilerini sisteme ekler.

Test ortamında birim testi için MSTest, veritabanı için MSSQL, performans testi için Visual Studio kullanıldı. Test planı ve test sonuç raporu Excel ve Word ile hazırlandı. Testleri yapabilmek için gereken beceriler SQL, C#, MSTest ve Excel-Word bilgisi olarak sıralandı.

Test takvimi şöyleydi: Ekim ayında test spesifikasyonu ve SRS hazırlandı. Ardından test planı oluşturuldu, test senaryoları hazırlandı, testler yürütüldü ve son olarak test raporu yazıldı. Üç ana test türü yapıldı: birim testi, kullanıcı testi ve performans testi.

5.2 Birim testi

Birim testleri Visual Studio'daki .NET birim test çatısıyla yazıldı. 12 birim testi vardı; alt durumlarıyla birlikte yaklaşık 30 test oldu ve hepsi geçti.

Grup iki pratik sorunla karşılaştı. Birincisi, uygulama Windows Forms olduğu için butonlar `private` tanımlıydı ve testlerden erişilemiyordu. Grup buton işlemlerini `public` fonksiyonlara taşıdı ve testleri bu fonksiyonlar üzerinden yazdı. İkincisi, uygulama veritabanı bağlantısı gerektiriyordu. İnternette "test yaparken gerçek veritabanına bağlanmayın" gibi öneriler vardı. Grup, veritabanında önemli bilgi bulunmadığı için veritabanını test ortamında da kullandı.

Örnek testler:

- **Kullanıcı ekleme:** test çalıştırıldı, kullanıcı eklendi; sonuç kullanıcılar tablosundan da doğrulandı.
- **Kullanıcı düzenleme (3 test):** var olan bir kullanıcının adını yenisiyle değiştirme; adı veritabanında kayıtlı bir adla değiştirme; alanları boş geçme. Boş geçilmeye izin verilmediği için üçüncü test de beklendiği gibi sonuçlandı.
- **Kullanıcı silme:** test kullanıcısı silindi ve silme veritabanından doğrulandı.

  Ders sonrası not: Butonları `public` yapmak testi mümkün kılar ama arayüz kodunu test için değiştirir. Büyük olasılıkla daha iyi bir çözüm, iş mantığını arayüzden ayrı bir sınıfa taşıyıp o sınıfı test etmektir. Gerçek veritabanı kullanmak da testleri birbirine bağımlı ve tekrarlanamaz hale getirebilir; ayrı bir test veritabanı ya da sahte nesneler (mock) tercih edilir ve her test kendi verisini hazırlayıp sonunda temizler.

Birim testi sırasında veritabanı tasarımında da eksikler bulundu. Tablolar biletler, firmalar, hizmet tipleri, koltuklar, kullanıcılar, müşteri, ödeme tipi, şehirler, seyahat ve ülkelerdir. Hizmet tipleri tablosunda hizmet tipi ID'si birincil anahtardır (primary key), ama firmalar tablosunda bu alan için yabancı anahtar (foreign key) tanımlanmamıştır. Veritabanında tetikleyici (trigger) ve fonksiyon da kullanılmamıştır. Bu yüzden bir hizmet tipi silindiğinde ona bağlı firma kayıtları silinmez ve tutarsızlık oluşur.

5.3 Kullanıcı testi

Kullanıcı testinin amacı, uygulamadaki hataları ortaya çıkarmak, ciddiyet düzeyine göre sınıflandırmak ve raporlamaktı. Testler iki gruba ayrıldı: SRS'teki use case senaryolarında olan testler ve senaryolarda olmayan, sıradan bir kullanıcının ekranı görür görmez deneyebileceği davranışlar.

Use case senaryolarında her modül için ana başarı senaryosu ve alternatif senaryolar (13A, 16A gibi numaralarla) uygulandı. Sonuçlar:

Modül	Sonuç
Bilet satışı	Ana ve alternatif senaryolar başarılı, %100
Sefer ekleme	2 testin ikisi başarılı, %100
Tatil ekleme	Modül hiç çalışmıyor; "kaydet" tuşuna basınca tatil kaydedilmiyor. 2 testin ikisi başarısız, %0. 1. derece (kritik) hata
Kullanıcı düzenleme	3A-3D alternatifleriyle 5 test başarılı, %100
Müşteri ekleme	Ana senaryonun 11 maddesi ve 11A alternatifi başarılı, %100

Modül	Sonuç
Firma düzenleme	5 test başarılı, %100
Kullanıcı ekleme	Ana senaryo ve 1A, 3A, 4A alternatifleri başarılı, %100
Yolculuk tipi ekleme	2 test başarılı, %100
Ödeme tipi ekleme	2 test başarılı, %100
Tatil rezervasyonu	Modül çalışmıyor; gerekli bilgiler girilmeden "ödeme yap"a basılınca da çalışmıyor. 2 test başarısız, %0
Firma ekleme	Ana senaryo ve 4A-4C alternatifleriyle 4 test başarılı, %100

Senaryo dışı testler 5 taneydi ve hiçbiri geçmedi (%0):

- Firma düzenleme modülünde firma seçmeden işlem yapmak: `null` dönüşü kontrol edilmediği için hata çıkar.
- Yanlışlıkla eklenen bir ödeme tipini silmek: ödeme tipi silme modülü yoktur; eksiklik olarak kaydedildi.
- Yolculuk tipi eklemede sıfır koltuk girmek: sistem kabul eder. Sıfır koltuklu bir yolculuk olamaz; 3. derece (minör) hata.
- Kalkış saati geçmiş bir sefere bilet satmak: örneğin saat beşte kalkmış bir sefere altıda, yedide bilet satılabilir. Programın akışını etkilemeyen bir mantık hatasıdır; 2. derece.
- Kullanıcı düzenleme modülünde kullanıcı seçmeden işlem yapmak: yine `null` kontrolü yoktur.

5.4 Performans testi

Performans testi, yazılımın beklenen iş yükü altında iyi performans gösterip göstermediğini ölçer. Bu uygulama yerel olduğu için beklenen iş yükü tek bir kullanıcıdır. Grup bunu gerçekçi bir kullanımla sınadı: beş üyeden biri sistem yöneticisi, dördü satış temsilcisi rolünde uygulamayı bir iş günü boyunca kullandı. Bu sırada uygulama release modunda çalıştırıldı ve Visual Studio Performance Profiler ile CPU ve bellek izlendi.

Ortalama bellek kullanımı yaklaşık 40 MB'tı. CPU'yu en çok etkileyen etkenler beklendiği gibi arayüzün oluşturulması ve veritabanı sorgularıydı. En çok çağrılan fonksiyonlar çekirdek (kernel) fonksiyonları, ardından arayüz fonksiyonlarıydı. CPU'yu en çok kullanan ekranlar kontrol paneli ve firma kaydetme gibi ekranlardı.

5.5 Sonuçların değerlendirilmesi

Toplam 75 test yapıldı: 30 birim testi, 40 kullanıcı testi ve 5 performans testi ($30+40+5=75$). Birim testlerinin hepsi geçti; kullanıcı testlerinde 9 hata bulundu.

Grubun yorumu şuydu: Uygulama çoğunlukla veritabanı işlemleri yapan bir masaüstü uygulaması olduğu için atomik yapılar, örneğin "müşteri ekle", mantıksal olarak doğru çalışıyor. Asıl sorun entegrasyondadır: bazı modüller hiç yazılmamış, bazı modüllerde `null` kontrolleri yapılmamış.

> **Ders sonrası not:** Bu bulgu test seviyelerinin neden birbirini tamamladığını gösterir. Bütün birim testleri geçse de modüller bir araya gelince ya da kullanıcı beklenmedik bir yol izleyince hata çıkabilir. Bu tür hatalar entegrasyon, sistem ve kullanıcı testlerinde ortaya çıkar. Birim testinin başarılı olması yazılımın doğru çalıştığı anlamına gelmez.

6. Beşinci Grup: Otopark Otomasyon Sistemi

6.1 Yazılımın tanıtımı ve gereksinimler

Test edilen yazılım Java ile yazılmış bir otopark otomasyon sistemidir. Ana ekranda sol üstte araç plakası girilir, ortada giriş ve çıkış saatleri için alanlar bulunur. Alt kısımda araç tipi

seçilir: resmî araç, normal araç ya da otoparka kayıtlı (üye) araç. Sağ tarafta kapasite, boş yer ve güncel kazanç bilgileri ile bunları güncelleme alanları yer alır. Ayrıca yeni kayıtlı araç ekleme, kayıtlı araçları ve otopark bilgilerini listeleme ve içerideki araçları listeleme butonları vardır.

SRS'teki fonksiyonel gereksinimler şunlardır:

- **Araç girişi:** giriş saati ve plakayla araç kaydedilir.
- **Araç çıkışı:** araç seçilir, çıkış saati ve plakayla çıkış yapılır. Otoparkta bulunmayan bir araç için uyarı verilir.
- **Üyelik kaydı:** başlangıç ve bitiş tarihleri alınır, tarihlerin geçerliliği doğrulanır, plakayla üyelik kaydı yapılır. Kayıttan vazgeçme ve hatalı bilgi bildirimi de vardır.
- **Sorgulama ve listeleme:** aracın park durumu sorgulanır; kayıtlı araçlar ve şu anda içeride bulunan araçlar listelenir.
- **Ücretlendirme:** çıkıştan sonra kalınan süre hesaplanır, saate göre ücret alınır ve güncel kazanca eklenir.
- **Otopark bilgileri:** boş yer, kapasite ve güncel kazanç görüntülenir.

Fonksiyonel olmayan gereksinimler arasında düşük kaynak kullanımı, düşük cevap süresi ve sistemin aynı anda tek kişi tarafından kullanılması sayıldı.

6.2 Test planı

Test edilen öğeler araç girişi, araç çıkışı ve üyelik kaydı gibi fonksiyonel gereksinimler ile güvenlik gibi fonksiyonel olmayan gereksinimlerdi. Test edilmeyen öğeler kullanıcı arayüzü bilgilendirmeleri, performans, platform ve tasarımdı. Testlerin amaçları şöyle sıralandı:

- Kullanıcı gereksinimlerinin karşılandığından emin olmak.
- Uygulamadaki hataları kullanıcıya sunulmadan önce düzeltmek.
- Kullanıcı memnuniyetini sağlamak.
- Ürün piyasaya sunulduktan sonra ortaya çıkabilecek yüksek maliyetli hataların önüne geçmek.

Test sürecindeki roller uygulama geliştirici (müşteri isteklerine göre uygulamayı geliştiren kişi), test yöneticisi (test adımlarının uygunluğunu kontrol eden ve gerçekleştiren kişi) ve müşteridir (uygulamayı tanımlayan ve beklentilerini açıkça belirten kişi).

Grup şelale (waterfall) yaklaşımını izledi: önce bütün gereksinimler gerçekleştirildi, sonra teste geçildi. Modül seviyesinde birim testi, uygulama seviyesinde sistem ve kullanıcı testi yapıldı. Tamamlanma kriterleri şunlardı: bütün testler çalıştırılmış olmalı, tespit edilen hatalar bir sonraki teste kadar düzeltilmeli ve testlerin en az %80'i başarılı olmalı. Fonksiyonel gereksinimler testlerle, fonksiyonel olmayan gereksinimler gözlemle doğrulandı. Bulunan hatalar ve değiştirilmesi gereken modüller bir belgeye yazıldı; bir sonraki testte bu hataların giderilip giderilmediği kontrol edildi.

Sistem Java masaüstü uygulaması olduğu için birim testinde JUnit, sistem testinde Winium kullanıldı. Test ortamı için en düşük gereksinim Windows 10'du. Test takviminde sırasıyla birim, sistem ve kullanıcı testi yer aldı.

6.3 Risk yönetimi

Test planında dört risk raporlandı:

Risk	Olasılık	Etki	Açıklama
Proje süresi	Yüksek	Büyük	İşlerin gecikmesi projenin tamamlanmasını uzatır.

Risk	Olasılık	Etki	Açıklama
Test ekibi	Orta	Büyük	Ekip farklı projelerde görev alırsa test faaliyetleri aksar. Çözüm iş planı yapmaktır.
Araçlar	Orta	Büyük	Testte kullanılan araçların yapılandırılması gerekir.
Test süresi	Orta	Büyük	Modüller zamanında kodlanmazsa testler gerekli zamanda tamamlanamaz.

Test raporunda birim testlerinin çoğunun başarıyla sonuçlandığı ama bazı modüllerde hata bulunduğu, sistem testinin Winium ile başarıyla tamamlandığı ve son kullanıcı testinde sorun çıkmadığı belirtildi. Grup, bazı hataların ortaya çıkarılmış olmasını hedeflenen başarıya ulaşıldığının göstergesi olarak yorumladı.

6.4 Birim testi

Birim testleri JUnit ile yazıldı. Grup JUnit'i kullanım kolaylığı ve internette çok kaynak bulunması nedeniyle seçtiğini, ayrıca JetBrains'in bir anketine göre Java geliştiricilerinin %83'ünün JUnit'i tercih ettiğini söyledi.

Test edilen işlemler şunlardı:

- **Araç girişi:** geçerli verilerle aracın veritabanına eklenmesi ve boş yer kapasitesinin güncellenmesi beklenir. Geçersiz veride işlem başarısız olmalıdır.
- **Araç çıkışı:** plaka ve çıkış saati girilir; aracın içerideki araçlar tablosundan çıkarılması beklenir. Yanlış veride işlem başarısız olmalıdır.
- **Park durumu sorgulama**, **üye araçlar** ve **üyelik kaydı** da birim testleriyle kontrol edildi.
- **Aynı plaka:** bir araç ilk eklendiğinde işlem başarılı olmalı; aynı plakalı araç tekrar eklenmek istendiğinde eklenmemelidir.

Yaklaşık 35 test yazıldı ve canlı çalıştırıldığında hepsi geçti. Testler sırasında iki hata fark edildi:

- **SQL injection:** plaka giriş alanı SQL injection saldırısına açıktı. Kullanıcı kendi yetki alanı dışında istediği SQL sorgusunu çalıştırabiliyor; örneğin hasılat bilgisini güncelleyebiliyor ya da tabloları silebiliyordu.
- **Geçmiş tarih:** araç girişinde güncel tarihten önceki bir tarihle kayıt yapılabiliyordu.

  Ders sonrası not: SQL injection açığı büyük olasılıkla kullanıcı girdisinin sorgu metnine doğrudan eklenmesinden kaynaklanır. Çözüm parametreli sorgudur; Java'da `PreparedStatement` ile girdi, sorgunun yapısından ayrı olarak gönderilir.

6.5 Sistem testi ve kapsam

Sistem testi, sistemin bütün öğeleri bir araya geldiğinde uyumlu çalıştığını doğrulamak için yapıldı. Araç Winium'du: Windows masaüstü uygulamalarını test etmek için kullanılan, Selenium tabanlı bir araçtır ve Java, JavaScript, Objective-C gibi farklı teknolojilerle kullanılabilir. Grup aracı, projede kullanılan teknolojilerle uyumlu olduğu ve istenen test durumlarını gerçekleştirebildiği için seçti.

İki sistem testi senaryosu vardı: araç girişi ve araç çıkışı. Her senaryonun adımları bir tabloda verildi; adımları Winium otomatik olarak uyguladı ve iki senaryoda da beklenen sonuçlar alındı. Bu testlerle uygulamanın işletim sistemiyle uyumu da sınanmış oldu.

Kapsam ölçümü (coverage) test sırasında kodun ne kadarının çalıştırıldığını belirler. Grup ek bir bağımlılık gerektirmediği için IntelliJ IDEA'nın yerleşik kapsam aracını kullandı. Sonuç %20 çıktı. Grup bunu, kodda testlerin hiç çalıştırmadığı çok fazla arayüz kodu bulunmasıyla açıkladı.

Hoca bu noktada sunumu kısaca övdü.

> **Hocanın vurgusu:** Sunum şimdiye kadar güzeldi. Özellikle kapsam (coverage) için de bir araç kullanılmış olması iyi olmuş.
>
> **Ders sonrası not:** Grup bu ölçümü "kapsam testi" diye adlandırdı. Kapsam büyük olasılıkla ayrı bir test türü değil, mevcut testlerin kodun ne kadarını çalıştırdığını gösteren bir ölçüttür. %20 gibi düşük bir değer, arayüz kodunun da sistem ya da arayüz testleriyle sınanması gerektiğini gösterir.

6.6 Kullanıcı testi ve uçtan uca gösterim

Kullanıcı testinde uygulamanın kullanılabilirliği son kullanıcıyla sınandı. Örnek olarak araç girişi test edildi: plaka, saat ve dakika girildikten sonra araç sisteme sorunsuz giriş yaptı. Sunumun sonunda grup bir uçtan uca (end-to-end) testi de canlı gösterdi: test uygulamayı kendisi açtı, araç girişini saat ve dakika bilgileriyle yaptı ve işlem başarıyla tamamlandı.

7. Altıncı Grup: Sherlock Komut Satırı Aracı

7.1 Yazılımın tanıtımı ve gereksinimler

Bu grup kendi yazdığı bir yazılımı değil, açık kaynaklı bir projeyi test etti. Sherlock, Python ile yazılmış, komut satırından (shell) çalışan bir araçtır. Verilen bir kullanıcı adını yüzlerce sosyal medya sitesinde arar ve bulunan profillerin bağlantılarını listeler. GitHub'da 15-20 bin yıldızı olan, yıllardır geliştirilen popüler bir projedir. Grup bu yüzden testlerde çok az hata bulduğunu söyledi.

Hoca gruptan önce gereksinim analizini kısaca vermesini, ardından planı, test durumlarını ve sonuçları anlatmasını istedi.

Gereksinim analizinden çıkan en önemli nokta şuydu: Siteler bir kullanıcı adının var olup olmadığını üç farklı yolla bildirir:

- HTTP durum kodu (status code),
- yanıt URL'si (response URL),
- hata mesajı (error message).

Bu yüzden grup her testi bu üç mekanizma için ayrı ayrı yazdı. Örneğin kullanıcı adının doğru bulunup bulunmadığı test edilecekse üç ayrı test gerekir. Fonksiyonel olmayan gereksinim ise hız ve eş zamanlılıktı: yüzlerce siteden veri sırayla çekilse çok yavaş olurdu, bu yüzden veri eş zamanlı çekilmelidir.

7.2 Test planı

Test planında şunlar test edildi:

- Arayüze doğru ve hatalı kullanıcı adı girdileri.
- Yanıt URL'siyle, HTTP durum koduyla ve hata mesajıyla bilgi veren sitelerden doğru verinin alınıp alınmadığı.
- Proxy ve Tor üzerinden sitelere bağlantı kurulup kurulamadığı.
- Çıktı raporları ve ayarlanan sitelerden veri alınıp alınamadığı.

Kalite hedefleri kullanıcı gereksinimlerinin tam karşılanması, hataların yayından önce bulunup düzeltilmesi, kullanıcı memnuniyetinin artırılması ve yayından sonra çıkabilecek hataların önlenmesiydi. Grup, yazılımın zaten yayımlanmış olmasının burada bir ayrıntı olduğunu not etti.

Test ortamı olarak Python 3 ya da üstü, internet bağlantısı ve kurulu bir Tor uygulaması gerekiyordu; kod PyCharm'da çalıştırıldı. Araçlar şunlardı: birim testi için Python `unittest`, sistem testi için Microsoft WinAppDriver, performans için PyCharm Profiler. Test yapacak kişilerin

bu araçları ve Python'u bilmesi eğitim ihtiyacı olarak yazıldı. Test çıktıları test planı, test durumları (test case), test kayıtları (log) ve test raporuydu. Risk olarak proje süresi, proje grubu ve kullanılan araçlar belirlendi; yalnız proje süresinde bir sıkıntı yaşandı ve giderildi.

7.3 Birim testi

13 birim testi yazıldı ve hepsi geçti. Örnekler:

- Doğru girilen bir kullanıcı adı sitelerde aramayı tetikliyor mu ve ekrana doğrulama mesajı veriyor mu?
- Parantez ya da kesme işareti gibi geçersiz karakterler içeren bir kullanıcı adı girilince hata mesajı veriliyor mu?
- Durum koduyla çalışan sitelerde arama doğru yapılıyor ve sonuç doğru yazdırılıyor mu? Yanlış ad için doğru sonuç veriliyor mu?
- Yanıt URL'si mekanizmasıyla çalışan sitelerde sonuç doğru raporlanıyor mu?
- Hazırlanan test verisi sitelere girildiğinde bütün sitelerden doğru dönüş alınıyor mu?

Gösterimde testlerin önceden hazırlanmış test verisini okuduğu ve sitelerde sırayla denediği görüldü; örneğin test verisinden bir site okunup yanıt URL'si üzerinden kullanıcı adının doğruluğu sınandı.

7.4 Sistem testi, performans ve kapsam

Sistem testinde sistem genel olarak sınandı; özellikle proxy ve Tor gibi aracı sistemler üzerinden bağlantının doğru çalışıp çalışmadığına bakıldı. Amaç, sistem dışarıdan bir başka sistemle birleştiğinde doğru çalışıp çalışmadığını görmekti.

Performans testinde PyCharm Profiler fonksiyonların çalışma sürelerini gösterdi: yeşil olanlar kabul edilebilir sürede, kırmızı olanlar daha uzun sürede çalışan fonksiyonlardı. Eş zamanlılık testinde aynı anda kaç siteden veri çekilebildiğine bakıldı; 19 farklı thread görüldü. Kapsam ölçümü, bir önceki grup gibi ama IntelliJ yerine PyCharm üzerinden yapıldı; toplam kapsam yaklaşık %70'ti.

> **Ders sonrası not:** WinAppDriver büyük olasılıkla Windows grafik arayüzlü uygulamaları otomatikleştirmek için kullanılır. Sherlock bir komut satırı aracı olduğu için sistem testi genelde programı bir betikle farklı girdilerle çalıştırıp çıktısını kontrol ederek yapılır.

8. Yedinci Grup: Flask ile Yazılmış Alışveriş Uygulaması

8.1 Durum ve yazılım

Bu grubun üç üyesi görev dağılımından sonra projeyi bıraktığı için çalışma iki kişiyle tamamlandı. Bu nedenle yalnız birim testi ve kullanıcı kabul testi (user acceptance test, UAT) yapılabildi. İlk seçilen uygulama ayağa kaldırılamadığı için benzer işlevli başka bir uygulamaya geçildi.

Test edilen uygulama Flask ile yazılmış küçük bir alışveriş uygulamasıdır. Kullanıcı e-posta ve parolayla giriş yapar, kayıt olur, profilini günceller, parolasını değiştirir, sepete ürün ekler ve çıkarır. Ödeme özelliği yoktur, bu yüzden ödemeyle ilgili test yapılmadı.

8.2 Kullanıcı kabul testi

Uygulama küçük olduğu için içindeki bütün işlevler kullanıcı üzerinde test edilebildi. Alfa testini (alpha test) iki geliştirici, beta testini (beta test) son kullanıcı olarak seçilen iki kişi yaptı. Kullanım ortamları Windows 10 ve macOS'tu. Test edilen işlevler şunlardı:

- **Giriş:** kayıtlı e-posta ve şifreyle giriş yapılır, kullanıcı uygun sayfaya yönlendirilir.
- **Kayıt:** kayıtlı olmayan kullanıcı formdaki zorunlu alanları doldurarak kayıt olur.
- **Parola değiştirme:** önce eski şifre doğrulanır, sonra yeni şifre kaydedilir.
- **Sepete ekleme ve çıkarma:** giriş yapmış kullanıcı ürün listesinden ürün seçip sepete ekler, sepetten çıkarır.
- **Profil güncelleme:** kullanıcı e-posta ve şifre dışındaki kişisel bilgilerini günceller.

Bütün testler geçti. Riskler şöyle sıralandı: zaman kısıtı yüzünden tamamlanamayan testler (olasılık orta, etki yüksek), deneyimsiz test ekibi (olasılık düşük, etki orta), yetersiz use case ve hata giderimi. Takım rollerinde iki kişi geliştirici ve alfa testçi, iki kişi beta testçiydi; UAT yönetiminden bir üye sorumluydu.

8.3 Birim testi

Birim testleri şunları kapsadı:

- **Kayıt (`test_register`):** e-posta ve parolayla kayıt yapılıyor mu? E-posta biçimine uymayan bir girdi ve boş parola reddedildi. Ancak aynı bilgilerle (aynı e-posta, parola, ad, soyad) ikinci kez kayıt olunabildiği görüldü. Bu, testlerde bulunan bir kusurdur.
- **Giriş ve çıkış:** doğru e-posta ve parolayla giriş yapılır. Doğru e-posta ve yanlış parola ya da yanlış e-posta ve doğru parolayla giriş yapılamaz.
- **Profil:** giriş yapılınca profil görüntülenir. Giriş yapılmadan doğrudan profil sayfasına gidilince ana sayfaya yönlendirilir.
- **Parola değiştirme:** eski parola doğru girilince parola değişir ve yeni parolayla giriş yapılabilir. Eski parola yanlışsa parola değişmez.
- **Ürünler:** bütün ürün sayfaları doğru görüntülenir.
- **Sepet:** sepet yalnız giriş yapmış kullanıcıya açıktır. Sepete eklenen ürün sepette görünür.

Hoca, diğer üyeler projeyi bıraktığı halde grubun yapabildiği kadarını tamamladığını söyleyerek teşekkür etti.

9. Sekizinci Grup: Instagram Otomatik Hesap Oluşturma Botu

9.1 Yazılım ve test planı

Test edilen yazılım Chrome ve Firefox için bir tarayıcı eklentisidir. Geçici (fake) bir e-posta adresi alır; ad-soyad, kullanıcı adı ve şifreyi doldurur, doğum tarihini seçer ve gelen doğrulama kodunu girerek kaydı tamamlar. Üç ana modülü vardır: kayıt (ad-soyad, kullanıcı adı, şifre), doğum tarihi seçimi ve geçici e-posta ile doğrulama kodu.

Test stratejisinde her test seviyesinin modülleri belirlendi:

- **Birim testi:** e-posta doğrulama, ad-soyad doğrulama, kullanıcı doğrulama ve şifre doğrulama.
- **Sistem testi:** performans testi altında yük, stres, dayanıklılık, ani yük (spike), hacim ve ölçeklenebilirlik testleri.
- **Kullanıcı testi:** kayıt işlemi, kayıt bilgilerinin girilmesi ve geçici e-posta doğrulaması.

Giriş ve çıkış kriterlerinde her test için başlangıç koşulları belirtildi; örneğin e-posta doğrulama doğru girişte doğru çalışıyor mu, yanlış girişte nasıl bir yanıt veriyor? Hatalar derecelendirildi. Risk yönetiminde sistemin çalışmasından ve ekip içinden doğabilecek durumlar ele alındı. Araçlar şunlardı: birim testi için Python standart kütüphanesi (`unittest`), sistem testi için Selenium, performans (yük ve stres) testi için JMeter. Eklenti farklı işletim sistemleri için sistem gereksinimleriyle birlikte tanımlandı ve her test için başlangıç-bitiş tarihli bir takvim yapıldı.

> **Ek bilgi (derste anlatılmadı):** Grubun saydığı performans testi türleri büyük olasılıkla şöyle ayrılır. Yük testi (load test) beklenen yük altındaki davranışı, stres testi (stress test) sistemin kapasite sınırını ve aşıldığında nasıl çöktüğünü ölçer. Dayanıklılık testi (endurance/soak test) uzun süreli yükte bellek sızıntısı gibi sorunları, ani yük testi (spike test) yükün aniden artıp azalmasına tepkiyi sınar. Hacim testi (volume test) büyük veri miktarıyla, ölçeklenebilirlik testi (scalability test) ise kaynak eklendikçe performansın nasıl değiştiğiyle ilgilenir.

9.2 Birim testi

Bot geçici e-postayı dışarıdaki bir servisten aldığı için çok az fonksiyon birim olarak test edilebildi. Test edilen şey, botun ürettiği değerin Instagram'ın ilgili giriş alanına aynen yazılıp yazılmadığıydı:

- Bot geçici e-postayı alır ve Instagram'ın e-posta alanına girer. Alandaki değer botun ürettiğiyle eşleşirse test "OK" döner; eşleşmezse "fail" döner ve hatanın yerini gösterir.
- Aynı kontrol rastgele üretilen ad-soyad ve kullanıcı adı için yapılır.
- Aynı kontrol bot tarafından üretilen şifre için yapılır.

9.3 Sistem testi

Uygulama yerelde çalışan bir tarayıcı eklentisi olduğu ve arkasında veritabanı gibi ağır bir sistem bulunmadığı için performans testlerinden yalnız yük testi yapılabildi. Test, Instagram adresine erişilip erişilemediğini kontrol eder.

Grup, Selenium'un test için geliştirilmiş bir çatı olduğunu ama performans ya da sistem testi için uygun olmadığını, bu iş için JMeter'ın önerildiğini söyledi. Yöntem şöyleydi: tarayıcıdaki adımlar Katalon Recorder eklentisiyle kaydedildi (alanlara yazma, butona basma), kayıt durdurulup dışa aktarıldı (export). Oluşan test örneği JMeter'a konup çalıştırıldı. JMeter sayfaya gidip yanıtı aldı ve sonucu bir sonuç ağacında (result tree) gösterdi; gösterimde sayfaya ulaşıldığı ve her şeyin "OK" olduğu görüldü.

> **Ders sonrası not:** Grubun "Selenium sistem testi için uygun değil" ifadesi tam doğru değil. Selenium büyük olasılıkla arayüz üzerinden fonksiyonel ve sistem testi için uygundur; uygun olmadığı alan yük ve performans testidir. Yük testi için JMeter gibi araçlar kullanılır.
>
> **Ders sonrası not:** Otomatik hesap açan botlar büyük olasılıkla Instagram'ın kullanım koşullarına aykırıdır. Gerçek bir projede bu tür yasal ve sözleşmesel sınırlar da gereksinim ve risk analizine girmelidir.

10. Dokuzuncu Grup: Android TV için Leanback Launcher

10.1 Yazılımın tanıtımı

Grup, Android TV cihazları için etkileşimli bir başlatıcı (launcher) arayüzü geliştirdi. Bu, televizyon ya da TV kutusu açılırken görülen ana ekrandır. Arayüz uygulamaları gruplar: kullanışlı araçlar (uygulama indirme gibi) ve diğer bütün uygulamalar. Kullanıcı favori uygulama seçebilir (favoriler en üstte görünür), indirdiği uygulamaları silebilir ve kumandadaki kırmızı, yeşil, sarı ve mavi tuşlara uygulama atayabilir. Hızlı ayarlar altında dil, tarih-saat, çevrimiçi güncelleme, sistem bilgisi, ekran ayarları, ağ ve Bluetooth ayarlarına kolay erişim vardır. Grup bütün dil dosyalarını da ekleyip tek tek test etti; renkli tuşlara basınca ilgili uygulamanın açılıp açılmadığını, ekran ayarı değiştirildiğinde gerçekten değişip değişmediğini kontrol etti.

Ayrıca "Clear Memory" adlı ek bir uygulama yazıldı. Kullanıcı birden fazla uygulamaya girdiğinde her biri belli bir bellek tutar ve bu bellek serbest bırakılmaz. Bu uygulama bir animasyonla belleği serbest bırakır (release) ve arayüze geri döner.

10.2 Test planı

Yazılım test faaliyetleri birim, entegrasyon ve sistem testlerinden oluştu. Test edilen modüller favori uygulama ekleme, uygulama silme ve uygulama ekleme modülleriydi.

- **Giriş kriterleri:** kodun yazılması, kod hatalarının gözden geçirilmesi, birim testlerinin yapılması, testlerin uygulanması ve hataların çözülmesi.
- **Çıkış kriterleri:** bütün kabul kriterlerinin sağlanması ve %95 olarak belirlenen hedef kalite seviyesine ulaşılması.

Hata yönetimi için bir Git platformu kullanıldı. Bulunan hatalar sisteme raporlayanlar tarafından eklenir ve hata derecesine göre etiketlenir: sistemin genel çalışmasını doğrudan etkileyen 1. derece hatalar "acil", genel çalışmayı etkilemeyip bazı işlevleri engelleyen 2. derece hatalar "önemli", görsel 3. derece hatalar "hata" etiketi alır.

Riskler şunlardı: geliştirme ekibinin başka projelerde çalışması yüzünden teste geçişin gecikmesi, test ekibinin başka projelerde görev alması yüzünden testin gecikmesi, modüllerin zamanında bitmemesi yüzünden teste başlanamaması ve test ortamının yetersizliği yüzünden

testlerin doğru yapılamaması. İlk üçü süre riskidir. Birim testlerini geliştiriciler, entegrasyon ve sistem testlerini test ekibi yaptı.

Program Java ile yazıldığı için birim testinde JUnit kullanıldı. Sistem testi için Appium tabanlı TestProject platformu ve Selendroid kullanıldı. Testler bir Android TV kutusunda ve bir telefonda çalıştırıldı. Grup telefonun istenen performansı vermediğini, ama herkeste TV kutusu bulunmadığı için telefonla test etmenin faydalı olduğunu söyledi. TestProject cihaza Appium yükler; Appium, Android cihaz ile bilgisayar arasında iletişimi sağlayan açık kaynaklı bir araçtır ve ekran görüntüsü alma, canlı gösterme gibi yetenekleri vardır.

10.3 Sistem testi: tekrarlı otomatik test

TestProject'te test senaryosu adım adım kaydedilir. Araç adımları kendisi uygular ve sonunda bir koşulu kontrol eder. Örnek senaryo: Chrome favorilere ekleniyor; test en sonda Chrome'un favoriler satırına gelip gelmediğini kontrol eder. Aynı test 50-60 kez çalıştırılıp kaç kez geçtiği ölçülerek favori ekleme işlevinde hata olup olmadığı anlaşılır. Test birden fazla cihazda aynı anda çalıştırılabilir.

İlk ölçümde favori ekleme testi yaklaşık 92 çalıştırmada %74 başarı verdi. Grup sebebi araştırdı ve buldu: arayüz, liste yüklenmeden yukarı kaymaya çalışıyordu. Düzeltmeden sonra araç testi arka arkaya 42 kez çalıştırdı ve hepsi başarılı oldu.

Canlı gösterimde başarı oranı %71 çıktı. Grup bunu gösterim sırasında cihazın bağlantısını kesmesine ve internet bağlantısının kopmasına bağladı. Araç her test için ayrı bir rapor üretir: hangi adımda başarısız olunduğunu (gösterimde beşinci adım) ve o andaki ekran görüntüsünü verir. Gösterimdeki ekran görüntüsü, uygulamanın kapatılıp başka bir ekrana geçildiğini gösteriyordu.

> **Ek bilgi (derste anlatılmadı):** Kod değişmeden bazen geçen, bazen kalan teste büyük olasılıkla kararsız test (flaky test) denir. Sebep çoğu zaman zamanlamadır; burada olduğu gibi liste yüklenmeden işlem yapılması tipik bir örnektir. Arayüz otomasyonunda sabit süreli beklemeler yerine, bir öğe görünene ya da yüklenene kadar bekleyen koşula bağlı (explicit) beklemeler kullanılması bu tür hataları azaltır. Aynı testi çok kez tekrarlayıp başarı oranını ölçmek, grubun yaptığı gibi, kararsızlığı ortaya çıkarmanın bir yoludur.

10.4 Performans

Başlatıcı açılırken paket adlarını, ikonları ve banner'ları okuduğu için CPU kısa süre yükselir, sonra düşüp sabitlenir. Grup bunun neredeyse bütün başlatıcılarda böyle olduğunu söyledi. Bellek kullanımı en fazla yaklaşık 64 MB'tı; grup bunu cihazla ilgili bütün bilgileri getiren bir uygulama için uygun buldu. Düşük sürümlü cihazlarda düşük FPS görülebiliyordu. Grup tasarımda sadeliği ve her yaştan kullanıcının kolay öğrenebilmesini hedeflediğini belirtti.

11. Tek Kişilik Proje: Depo Stok Yönetim Sistemi

Son sunumu tek kişilik bir proje yaptı. Yazılım bir depo stok yönetim sistemidir: ürün ekleme ve silme, stok girişi, güncellemesi ve silmesi yapılır; ürün ve stok listeleri rapor olarak dışarı aktarılabilir. Proje analiz, tasarım, gerçekleştirme, test ve bakım olmak üzere beş adıma bölündü; ticari bir depo yönetim yazılımı örnek alındı.

Test faaliyetleri dört aşamaya bölündü. İncelemede projenin 5 arayüz ekranı gereksinimlere göre kontrol edildi ve butonların işlevselliği sınandı. Test edilmesi gereken modüller şunlardı:

- **Ürün modülü:** ürün ekleme ve silme.
- **Stok modülü:** sistemdeki ürün listesinden ürün girişi, güncelleme ve silme.
- **Raporlama:** ürün ve stok listelerinin uygulama içinde ve dışarıya alınabilmesi.

Hatalar diğer gruplar gibi üç dereceye ayrıldı; 1. derece hatalar bulunduğu anda çözüldü. Kullanıcı testinde örneğin ürün kodunun sayı olarak girilmesi gerektiği kontrol edildi ve eklenen ürünün veritabanına yazıldığı SQL üzerinden doğrulandı. Kullanıcı testinde bir hata bulundu.

Birim testi Visual Studio'nun kendi birim test aracıyla ve yazılan doğrulama (validation) metotlarıyla yapıldı; iki birim testi vardı. Sistem testi olarak bir kaynak izleme aracıyla iki farklı bilgisayarda işlemci ve bellek kullanımı izlendi.

Hoca sunumun sonunda şu değerlendirmeyi yaptı: test adımlarını, özellikle sistem testi ve birim testi adımlarını tam takip edemediğini, sunumun genel kaldığını ve yalnız kullanıcı testinin gösterildiğini söyledi. Ardından birim testinin ne kullanılarak yapıldığını ve sistem testinin nasıl oluşturulduğunu sordu (bkz. Örnek Sorular). Tek kişi olmasına rağmen bir şeyler toparladığı için teşekkür etti.

> **Ders sonrası not:** Bu projede sistem testi yalnız performans ölçümü olarak anlatıldı; sunan kişi de "sistem testi geniş bir alan" dedi. Sistem testi büyük olasılıkla daha geniştir: bütünleşik sistemin fonksiyonel ve fonksiyonel olmayan gereksinimlerini uçtan uca doğrular. Kullanıcının giriş yapıp ürün ekleyip rapor alması gibi uçtan uca senaryolar da sistem testine girer.

12. Sunumlardan Çıkan Ortak Noktalar

12.1 Grupların ortak hata derecelendirmesi

Grupların çoğu aynı üç dereceli hata sınıflandırmasını kullandı:

Derece	Tanım	Örnek etiket (9. grup)
1. derece	Sistemin genel çalışmasını doğrudan etkiler	acil
2. derece	Sistemi durdurmaz ama bir işlevin çalışmasını engeller	önemli
3. derece	Çalışmayı etkilemez, görsel bir kusurdur	hata

Dördüncü grubun örnekleri bu sınıflandırmayı somutlaştırır: tatil modülünün hiç çalışmaması 1. derece, geçmiş saatli sefere bilet satılabilmesi 2. derece, sıfır koltuklu yolculuk tipinin kabul edilmesi 3. derece olarak sınıflandırıldı.

12.2 Giriş ve çıkış kriterleri

Giriş kriteri testin başlaması için sağlanması gereken koşullardır: gerekli alanların doldurulmuş olması, kodun yazılmış ve gözden geçirilmiş olması, birim testlerinin tamamlanmış olması gibi. Çıkış kriteri testin bitmiş sayılması için gereken koşullardır. Gruplardaki örnekler: bütün testlerin çalıştırılmış olması, testlerin en az %80'inin başarılı olması, bütün derecelerdeki hataların çözülmüş olması, %95 hedef kalite seviyesine ulaşılması ve bütün test faaliyetlerinin raporlanmış olması.

12.3 Kullanıcı testinin iki yüzü

Kullanıcı testinde use case senaryoları test edilir: ana başarı senaryosu ve alternatif senaryolar. Dördüncü grup buna, senaryoda olmayan ama sıradan bir kullanıcının deneyebileceği davranışları ekledi ve bu ikinci gruptaki 5 testin hiçbiri geçmedi. Yedinci grup ise alfa testini (geliştirici ekip) ve beta testini (gerçek son kullanıcılar) ayırdı. Birinci grup kullanıcı görüşünü bir anketle topladı.

12.4 Gruplarda kullanılan araçlar

Amaç	Araçlar
Birim testi	NUnit, MSTest, JUnit, Django `TestCase`, Python `unittest`

Amaç	Araçlar
Yük ve performans	Apache JMeter, Locust, Windows Performance Monitor, Visual Studio Performance Profiler, PyCharm Profiler
Masaüstü arayüzde sistem testi	Winium, WinAppDriver
Web arayüzünde sistem testi	Selenium, Katalon Recorder
Android arayüzünde sistem testi	TestProject (Appium tabanlı), Selendroid
Kapsam (coverage)	IntelliJ IDEA, PyCharm

> **Ders sonrası not:** Altyazıda bazı araç ve ürün adları bozuk geçer. Doğruları: "Limbeck Launcher" Leanback Launcher, "Vinyum" Winium, "G-metre / G-Metri" JMeter, "katalona rekorder" Katalon Recorder, "test projekti / Test Project ION" TestProject.io, "Sel Android" Selendroid, "Paycham / Payçam" PyCharm, "Intelli-City" IntelliJ IDEA, "SIM Builder" Scene Builder, "Maben" Maven, "bulletin test case" Django'nun yerleşik `TestCase` sınıfı.

Örnek Sorular

Bu derste ekran karesi yoktur. Aşağıdaki sorular hocanın sunumlar sırasında gruplara sözlü olarak sorduğu sorulardır.

Soru 1 (derste sözlü, 00:23:05): Performans testinde hangi aracı (tool) kullandınız?

> **Cevap (derste verildi):** Birinci grup, uygulama bir masaüstü uygulaması olduğu için Windows Performance Monitor aracını kullandı. Bu araçla CPU yükü, disk erişimi ve thread sayıları ölçüldü; sonuçlar düşük çıktı.

Soru 2 (derste sözlü, 00:37:01): Birim testini yaptığınız ürün, yani araç (tool) nedir?

> **Cevap (derste verildi):** İkinci grup önce test edilen ürünü (insan kaynakları sistemini) anlatmaya başladı; hoca, sorulanın ürün değil test aracı olduğunu belirtti. Cevap JUnit'ti. Örneğin `DBUtils` test sınıfı veritabanı bağlantısını kontrol ediyordu; 56 test yazılmıştı. Ders bağlamındaki çıkarım şudur: Bir test sunumunda her test seviyesi için kullanılan araç açıkça adlandırılmalıdır.

Soru 3 (derste sözlü, 00:38:22): Sistem testini yaptığınız araç hakkında bir iki cümle söyler misiniz?

> **Cevap (derste verildi):** İkinci grup sistem testini açık kaynaklı Apache JMeter ile yaptı. Grup bu araçla uygulamanın açılış süresini, açılışta harcanan bellek ve CPU miktarını görebildiklerini söyledi. Sonuçlar raporda verildi. Bu ölçümlerin JMeter'ın asıl kullanım alanıyla ilişkisi için bkz. Bölüm 3.4'teki ders sonrası not.

Soru 4 (derste sözlü, 02:31:50): Birim testini ne kullanarak yaptın?

> **Cevap (derste verildi):** Tek kişilik projede birim testi Visual Studio'nun kendi birim test aracıyla yapıldı. Buna ek olarak kodun içine bir doğrulama (validation) metodu yazıldı; kullanıcı hatalı ya da farklı bir bilgi girdiğinde metot bunu anında kullanıcıya döndürüyordu. Toplam iki birim testi vardı.

Soru 5 (derste sözlü, 02:32:27): Sistem testini nasıl oluşturdun?

> **Cevap (derste verildi):** Sunan kişi sistem testinin geniş bir alan olduğunu, kendi projesini performans testine tabi tuttuğunu söyledi: uygulamanın kullandığı işlemci ve bellek miktarına bir kaynak izleme aracıyla iki farklı bilgisayarda baktı. Hoca bu sunumda birim ve sistem testi adımlarını takip edemediğini belirtmişti. Sistem testinin kapsamı için bkz. Bölüm 11'deki ders sonrası not.

Dersin Toparlaması

Bu derste dokuz grup ve tek kişilik bir proje, dönem boyunca teorik olarak görülen test planı ve test sonuç raporu yapısını kendi projelerine uyguladı. Sunumlar birim testi için farklı dillerde farklı çatıların (NUnit, MSTest, JUnit, Django `TestCase`, Python `unittest`), sistem testi için

arayüz otomasyon araçlarının (Winium, WinAppDriver, Selenium, Katalon Recorder, TestProject, Selendroid), performans ve yük testi için ölçüm araçlarının (JMeter, Locust, Performance Monitor, Visual Studio ve PyCharm profilleyicileri) nasıl kullanıldığını gösterdi. Gruplar ortak bir üç dereceli hata sınıflandırması, giriş ve çıkış kriterleri, risk tabloları ve kullanıcı testinde ana ve alternatif senaryolar kullandı. Bulgulardan özellikle ikisi öğreticiydi: Dördüncü grupta bütün birim testleri geçerken kullanıcı testlerinde 9 hata çıkması, test seviyelerinin birbirini tamamladığını; dokuzuncu grupta aynı testin tekrarlanarak %74 başarıdan %100'e çıkarılması, zamanlamaya bağlı kararsız testlerin nasıl bulunup giderileceğini gösterdi.

Hocanın geri bildirimlerinden çıkan ana dersler şunlardır: Proje tanımı ve gereksinim analizi kısa tutulmalı, use case'lerden yalnız birkaç örnek verilmelidir. Güvenilirlik ile güvenlik karıştırılmamalıdır; kimlik doğrulama bir güvenlik gereksinimidir. Her test seviyesinde kullanılan araç, ürünün değil test aracının adıyla açıkça söylenmelidir. Kapsam ölçümü gibi ek bir aracın kullanılması olumlu karşılanır. Birim ve sistem testi adımları izlenebilir biçimde gösterilmelidir; yalnız kullanıcı testine dayanan genel bir sunum yeterli değildir. Bu ders dönemin son dersiydi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.